

Desarrollo de software para el cuidado de la salud

Ruta práctica desde los fundamentos hasta sistemas avanzados

2026-07-31

Tabla de contenidos

Propósito del tutorial	1
Cómo estudiar esta ruta	2
El proyecto integrador: SaludLocal	3
Antes del código: entender el problema	3
Nivel 0. Preparar un entorno reproducible	4
Nivel 1. Fundamentos de programación	6
Nivel 2. Calidad del código y trabajo colaborativo	7
Nivel 3. API, persistencia y diseño modular	11
Nivel 4. Software preparado para operar	13
Nivel 5. Ingeniería de software para salud digital	15
Nivel 6. IA, sistemas avanzados y operación continua	17
Tendencias actuales que vale la pena comprender	19
Efectividad y eficiencia del desarrollador	20
Plan de estudio de 40 semanas	21
Portafolio que demuestra capacidad	22
Lista final de verificación	22
Fuentes y documentación recomendada	23

Desarrollar software no consiste en memorizar lenguajes o acumular *frameworks*. Consiste en transformar una necesidad real en un sistema que pueda entenderse, verificarse, mantenerse y mejorar con el tiempo.

En salud, esta responsabilidad es mayor. Un error no solo afecta una pantalla: puede retrasar un proceso, ocultar información relevante o inducir una decisión equivocada. Por esta razón, aprenderemos programación, pero también requisitos, pruebas, seguridad, interoperabilidad, documentación, operación y evaluación de riesgos.

Propósito del tutorial

Esta guía propone una ruta desde el nivel principiante hasta el avanzado. El objetivo no es estudiar cada tecnología disponible. El objetivo es construir una base sólida, reconocer qué problema resuelve cada herramienta y usarla cuando aporte valor.

Al finalizar la ruta, el estudiante estará en capacidad de:

- convertir una necesidad de salud en requisitos y criterios de aceptación;
- construir aplicaciones con Python, control de versiones y entornos reproducibles;
- diseñar pruebas unitarias, de integración, de seguridad y de aceptación;
- desarrollar una API, persistir datos y documentar sus contratos;
- automatizar controles de calidad mediante integración continua;
- aplicar principios de privacidad, trazabilidad y seguridad desde el diseño;
- reconocer el papel de HL7 FHIR y DICOMweb en la interoperabilidad clínica;
- desplegar, observar y mantener un sistema en condiciones controladas;
- utilizar asistentes de inteligencia artificial con verificación humana; y

- decidir cuándo una tendencia mejora el sistema y cuándo solo agrega complejidad.

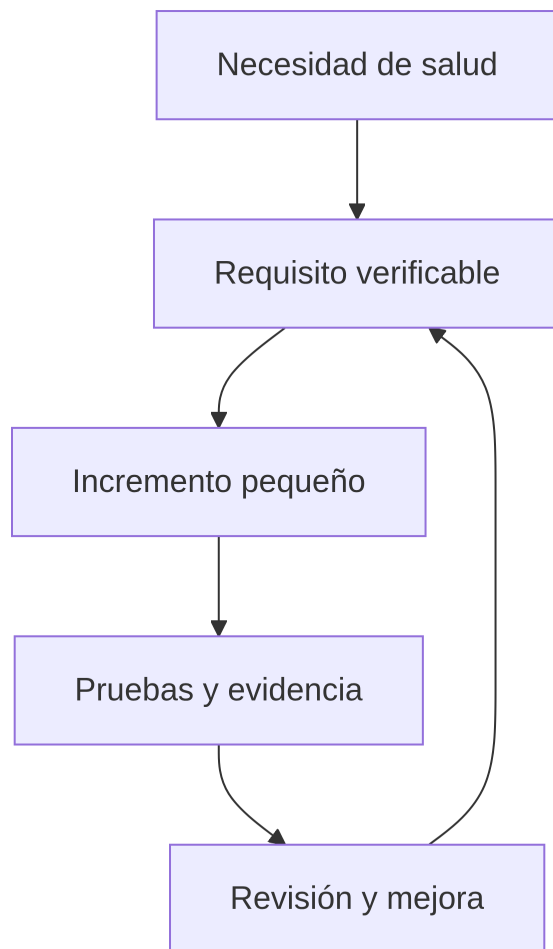
! Alcance educativo

El proyecto de esta guía utiliza datos sintéticos y reglas técnicas simuladas. No diagnostica, no recomienda tratamientos y no debe conectarse a historias clínicas reales. Un sistema destinado al uso clínico requiere validación con profesionales de salud, gestión formal de riesgos, controles institucionales y revisión de la regulación aplicable.

Cómo estudiar esta ruta

La ruta completa está pensada para **40 semanas**, con una dedicación aproximada de **8 horas por semana**. Si se dispone de más tiempo, puede comprimirse. Reducir el número de semanas no significa omitir pruebas, documentación o seguridad.

El ciclo de trabajo será siempre el mismo:



Una distribución semanal razonable es la siguiente:

Actividad	Proporción	Resultado esperado
Estudiar un concepto	20 %	Notas breves y preguntas concretas

Actividad	Proporción	Resultado esperado
Implementar	45 %	Un incremento pequeño que funcione
Probar y revisar	25 %	Evidencia de calidad y correcciones
Documentar y reflexionar	10 %	Decisiones, límites y siguiente paso

La regla central es sencilla: **cada concepto debe producir una evidencia**. Esa evidencia puede ser una prueba que pasa, una decisión documentada, una demostración reproducible o una explicación técnica breve.

El proyecto integrador: SaludLocal

Durante toda la ruta construiremos **SaludLocal**, un sistema educativo para registrar y consultar mediciones biomédicas sintéticas. El proyecto crecerá por etapas, sin ser reemplazado en cada nivel.

El sistema permitirá:

- registrar participantes mediante identificadores anónimos;
- almacenar mediciones sintéticas con fecha, unidad y procedencia;
- verificar rangos técnicos configurables sin emitir interpretaciones clínicas;
- consultar el historial y generar resúmenes descriptivos;
- exponer funciones mediante una API;
- registrar eventos de auditoría sin incluir datos sensibles en los registros;
- preparar una representación interoperable; y
- observar errores, tiempos de respuesta y disponibilidad.

Nivel	Incremento del proyecto	Evidencia principal
0. Preparación	Repositorio y entorno reproducible	Otra persona puede iniciar el proyecto
1. Fundamentos	Programa de línea de comandos	Entradas válidas e inválidas se manejan bien
2. Calidad	Pruebas, tipos, formato y documentación	El control automático detecta defectos
3. Aplicación web	API y base de datos	Contrato OpenAPI y pruebas de integración
4. Producción	Contenedor, seguridad y observabilidad	Despliegue repetible y diagnóstico de fallos
5. Salud digital	Interoperabilidad y gestión de riesgos	Trazabilidad entre necesidad, riesgo y prueba
6. Avanzado	IA y operación del ciclo de vida	Evaluación, monitoreo y plan de cambio

Antes del código: entender el problema

Un proyecto de salud no debe comenzar con la selección de una tecnología. Primero se define qué decisión o proceso se quiere apoyar.

Use estas preguntas:

1. ¿Quién utilizará el sistema?
2. ¿Qué tarea necesita realizar?

3. ¿Qué información necesita y cuál no debería recopilarse?
4. ¿Qué puede ocurrir si el sistema falla, se demora o entrega un dato incorrecto?
5. ¿Qué evidencia demostraría que la solución funciona?
6. ¿Quién conserva la responsabilidad sobre la decisión final?

De una necesidad a un requisito

Una necesidad amplia como “necesitamos mejorar el seguimiento” no se puede probar. Debe convertirse en una afirmación verificable.

Requisito funcional: el sistema debe permitir registrar una medición sintética asociada a un identificador anónimo, una fecha, un parámetro, un valor y una unidad.

Criterio de aceptación: cuando todos los campos sean válidos, el sistema debe guardar la medición y devolver su identificador; si falta un campo obligatorio, debe rechazar la solicitud con un mensaje comprensible.

El requisito describe **qué** debe ocurrir. El criterio de aceptación permite comprobarlo. La implementación explica **cómo** se logró.

Requisitos no funcionales

Los requisitos no funcionales expresan condiciones de calidad:

Dimensión	Pregunta útil	Ejemplo educativo
Seguridad	¿Quién puede realizar cada acción?	Solo un rol autorizado puede exportar datos
Privacidad	¿Qué información es estrictamente necesaria?	No se almacenan nombres ni documentos
Rendimiento	¿Cuánto puede tardar una operación?	El 95 % de consultas tarda menos de 500 ms
Disponibilidad	¿Qué interrupción es tolerable?	El entorno de práctica se recupera en 30 min
Usabilidad	¿Puede el usuario reconocer y corregir un error?	El mensaje identifica el campo inválido
Mantenibilidad	¿Puede modificarse sin romper lo existente?	Las reglas tienen pruebas automatizadas
Trazabilidad	¿Puede conocerse el origen de un dato?	Cada medición registra fuente y fecha

Nivel 0. Preparar un entorno reproducible

Meta: crear un proyecto que pueda ejecutarse en otro equipo sin reconstruir manualmente su entorno.

Duración sugerida: semanas 1 y 2.

Herramientas mínimas

Herramienta	Uso	Criterio de selección
Python	Lenguaje principal	Ecosistema amplio para web, datos e IA
uv	Python, entorno y dependencias	Flujo rápido y archivo de bloqueo reproducible

Herramienta	Uso	Criterio de selección
Git	Historial y colaboración	Permite revisar y recuperar cambios
VS Code o editor equivalente	Edición y depuración	Debe facilitar terminal, pruebas y Git
Quarto	Documentación técnica	Integra texto, código, tablas y diagramas

En proyectos de salud conviene preferir versiones estables y soportadas. En esta guía se utiliza Python 3.13 como base conservadora; una versión más reciente puede adoptarse después de verificar la compatibilidad de todas las dependencias.

Crear el proyecto

```
mkdir saludlocal
cd saludlocal

git init
uv init --python 3.13
uv add pydantic fastapi "uvicorn[standard]" sqlalchemy
uv add --dev pytest pytest-cov ruff pyright httpx

uv run python --version
uv lock
```

La documentación oficial de [uv](#) incluye métodos de instalación para Linux, macOS y Windows. Antes de ejecutar un instalador descargado, conviene revisar su procedencia y las instrucciones oficiales.

Estructura inicial

```
saludlocal/
  src/
    saludlocal/
      __init__.py
      models.py
      services.py
      main.py
  tests/
    test_services.py
  docs/
    decisiones/
  data/
    synthetic/
  .gitignore
  pyproject.toml
  uv.lock
  README.md
```

No se necesita una arquitectura compleja al comienzo. Una estructura pequeña y clara facilita aprender el dominio. La separación en capas se hará cuando el proyecto tenga razones concretas para necesitarla.

Primer registro de cambios

```
git status
git add pyproject.toml uv.lock README.md src tests
git commit -m "Inicializar proyecto SaludLocal"
```

Un buen *commit* representa una unidad de trabajo pequeña, coherente y revisable. Mensajes como **cambios** o **actualización** no explican qué ocurrió.

Criterio para avanzar

- ☐ El proyecto inicia desde una carpeta nueva con `uv sync`.
- ☐ Las dependencias están declaradas y bloqueadas.
- ☐ El repositorio no contiene contraseñas, llaves ni datos reales.
- ☐ El `README.md` explica propósito, alcance y forma de ejecución.

Nivel 1. Fundamentos de programación

Meta: expresar reglas simples mediante datos, funciones, condiciones, colecciones y manejo de errores.

Duración sugerida: semanas 3 a 8.

Conceptos que deben dominarse

- valores, variables y tipos;
- operadores y expresiones;
- condicionales;
- ciclos y colecciones;
- funciones con entradas y salidas claras;
- módulos e importaciones;
- excepciones esperadas;
- fechas, zonas horarias y unidades; y
- lectura básica de mensajes de error.

No es suficiente obtener una salida correcta una sola vez. Debe entenderse qué entradas acepta la función, qué devuelve y qué ocurre ante un dato inválido.

Primera regla de calidad de datos

El siguiente código verifica un rango **técnico configurable**. No interpreta el estado de una persona.

```
from dataclasses import dataclass
from datetime import datetime, timezone

@dataclass(frozen=True, slots=True)
class Medicion:
    participante_id: str
    parametro: str
    valor: float
    unidad: str
    registrada_en: datetime

def crear_medicion(
    participante_id: str,
```

```

parametro: str,
valor: float,
unidad: str,
minimo_tecnico: float,
maximo_tecnico: float,
) -> Medicion:
    if not participante_id.strip():
        raise ValueError("El identificador del participante es obligatorio")

    if minimo_tecnico > maximo_tecnico:
        raise ValueError("El rango técnico está invertido")

    if not minimo_tecnico <= valor <= maximo_tecnico:
        raise ValueError("El valor está fuera del rango técnico configurado")

    return Medicion(
        participante_id=participante_id.strip(),
        parametro=parametro.strip().lower(),
        valor=valor,
        unidad=unidad.strip(),
        registrada_en=datetime.now(timezone.utc),
    )

```

Qué debe observarse en el código

- La función tiene una responsabilidad principal.
- Los nombres expresan intención.
- La validación ocurre antes de construir el objeto.
- La fecha se registra en UTC.
- El objeto es inmutable después de crearse.
- El mensaje explica el problema sin revelar información sensible.

Ejercicio del nivel

Construya una interfaz de línea de comandos que reciba una medición, la valide y la muestre como JSON. Incluya al menos cinco entradas de prueba:

1. una entrada válida;
2. un identificador vacío;
3. un valor fuera del rango técnico;
4. un rango invertido; y
5. una entrada que no pueda convertirse a número.

Criterio para avanzar

- ☐ Puede explicar cada línea de la función.
- ☐ Diferencia un error esperado de un defecto del programa.
- ☐ Divide una tarea grande en funciones pequeñas.
- ☐ Maneja fechas, unidades y valores ausentes de forma explícita.
- ☐ No utiliza datos clínicos reales para practicar.

Nivel 2. Calidad del código y trabajo colaborativo

Meta: detectar defectos antes de que lleguen a otra persona o a otro entorno.

Duración sugerida: semanas 9 a 15.

Cuatro controles complementarios

Control	Pregunta que responde	Herramienta sugerida
Formato	¿El código mantiene una forma consistente?	Ruff Formatter
Análisis estático	¿Existen errores o prácticas problemáticas visibles?	Ruff
Tipos	¿Las entradas y salidas son compatibles?	Pyright
Pruebas	¿El comportamiento cumple lo esperado?	pytest

Ningún control reemplaza a los demás. Un programa puede estar bien formateado y producir un resultado incorrecto.

Primera prueba automatizada

```
import pytest

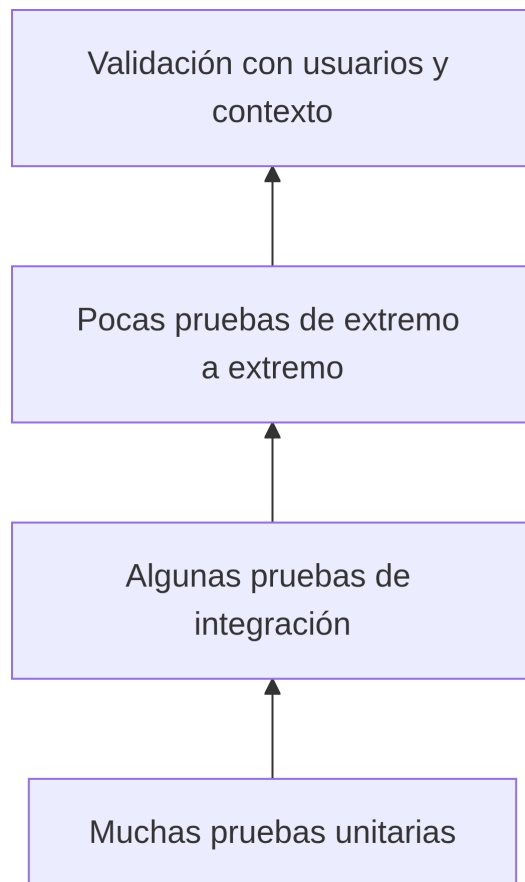
from saludlocal.services import crear_medicion

@pytest.mark.parametrize(
    ("valor", "minimo", "maximo"),
    [
        (0.0, 0.0, 10.0),
        (5.5, 0.0, 10.0),
        (10.0, 0.0, 10.0),
    ],
)
def test_crear_medicion_acepta_limites_y_valores_intermedios(
    valor: float,
    minimo: float,
    maximo: float,
) -> None:
    medicion = crear_medicion(
        participante_id="P-001",
        parametro="senal_simulada",
        valor=valor,
        unidad="u.a.",
        minimo_tecnico=minimo,
        maximo_tecnico=maximo,
    )

    assert medicion.valor == valor

def test_crear_medicion_rechaza_identificador_vacio() -> None:
    with pytest.raises(ValueError, match="identificador"):
        crear_medicion("", "senal_simulada", 5.0, "u.a.", 0.0, 10.0)
```

Pirámide práctica de pruebas



- **Prueba unitaria:** verifica una regla pequeña y aislada.
- **Prueba de integración:** comprueba que dos componentes colaboran correctamente.
- **Prueba de extremo a extremo:** recorre un flujo completo.
- **Validación de contexto:** determina si el sistema resuelve la necesidad real. No puede automatizarse por completo.

La cobertura de código es un indicador auxiliar. Alcanzar 80% no demuestra que las pruebas sean útiles. En software de salud importan especialmente los casos límite, los modos de fallo y la trazabilidad entre riesgo, requisito y prueba.

Flujo corto con Git

```
git switch -c feature/validar-mediciones

uv run ruff format .
uv run ruff check .
uv run pyright
uv run pytest --cov=saludlocal

git add src tests
git commit -m "Validar rangos técnicos de las mediciones"
```

Antes de integrar un cambio, otra persona debería poder responder:

- ¿Qué problema resuelve?
- ¿Qué archivos modifica?
- ¿Qué riesgo introduce?
- ¿Qué pruebas demuestran el comportamiento?
- ¿Cómo se revierte si falla?

Integración continua

La integración continua ejecuta los controles cada vez que se propone un cambio. Este ejemplo usa versiones principales para facilitar la lectura. En un entorno regulado conviene fijar acciones mediante identificadores completos y actualizarlas mediante un proceso revisado.

```
name: quality

on:
  push:
  pull_request:

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Obtener el código
        uses: actions/checkout@v7

      - name: Instalar uv
        uses: astral-sh/setup-uv@v9
        with:
          enable-cache: true

      - name: Sincronizar el entorno
        run: uv sync --locked

      - name: Verificar formato y reglas
        run: uv run ruff check .

      - name: Verificar tipos
        run: uv run pyright

      - name: Ejecutar pruebas
        run: uv run pytest --cov=saludlocal --cov-report=term-missing
```

Criterio para avanzar

- ☐ Cada regla importante tiene pruebas normales, límite e inválidas.
- ☐ La rama principal se mantiene ejecutable.
- ☐ Los cambios se integran mediante revisión.
- ☐ El entorno automático usa el archivo de bloqueo.
- ☐ Un fallo del control impide integrar el cambio.

Nivel 3. API, persistencia y diseño modular

Meta: convertir la lógica local en un servicio documentado, persistente y verificable.

Duración sugerida: semanas 16 a 23.

Separar responsabilidades

Cuando el proyecto crece, conviene separar el dominio de los detalles externos.

```
src/saludlocal/
  domain/          # Reglas y conceptos del problema
    entities.py
    errors.py
  application/     # Casos de uso
    services.py
  infrastructure/  # Base de datos y servicios externos
    database.py
    repositories.py
  api/             # Transporte HTTP
    schemas.py
    main.py
```

Esta estructura no busca cumplir una moda arquitectónica. Busca que una regla de negocio pueda probarse sin iniciar un servidor ni conectarse a una base de datos.

Contrato de entrada con Pydantic

```
from datetime import datetime

from pydantic import BaseModel, Field

class MedicionCreate(BaseModel):
    participante_id: str = Field(min_length=1, max_length=64)
    parametro: str = Field(min_length=1, max_length=80)
    valor: float
    unidad: str = Field(min_length=1, max_length=24)
    registrada_en: datetime
    fuente: str = Field(min_length=1, max_length=80)

class MedicionRead(MedicionCreate):
    id: str
```

Primer punto de acceso con FastAPI

```
from typing import Annotated

from fastapi import Depends, FastAPI, status

from saludlocal.api.schemas import MedicionCreate, MedicionRead
from saludlocal.application.services import MedicionService

app = FastAPI()
```

```

    title="SaludLocal API",
    version="0.1.0",
    description="API educativa con datos sintéticos.",
)

def get_service() -> MedicionService:
    return MedicionService()

@app.post(
    "/mediciones",
    response_model=MedicionRead,
    status_code=status.HTTP_201_CREATED,
)
def crear_medicion(
    entrada: MedicionCreate,
    service: Annotated[MedicionService, Depends(get_service)],
) -> MedicionRead:
    return service.crear(entrada)

```

FastAPI genera un contrato OpenAPI a partir de los tipos, pero el contrato debe revisarse. Una API bien documentada también define errores, permisos, límites, idempotencia, versión y política de cambios.

Decisiones de persistencia

Para iniciar, SQLite es suficiente. Para trabajo concurrente o despliegue, PostgreSQL suele ser una opción más apropiada. El cambio de motor no elimina la necesidad de diseñar el modelo de datos.

Registre de forma explícita:

- identificadores internos no predecibles;
- fecha del evento y fecha de recepción;
- unidad y sistema de unidades;
- procedencia del dato;
- versión de la regla que procesó el dato;
- estado de validez; y
- eventos de creación, corrección y anulación.

Nunca modifique silenciosamente un dato clínicamente relevante. Una corrección debe conservar la trazabilidad del valor anterior, el motivo y la identidad autorizada que realizó el cambio.

Prueba de integración de la API

```

from fastapi.testclient import TestClient

from saludlocal.api.main import app

client = TestClient(app)

def test_crear_medicion_devuelve_201() -> None:
    respuesta = client.post(
        "/mediciones",
        json={

```

```

        "participante_id": "P-001",
        "parametro": "senal_simulada",
        "valor": 5.2,
        "unidad": "u.a.",
        "registrada_en": "2026-07-31T13:00:00Z",
        "fuente": "simulador-docente",
    },
)

assert respuesta.status_code == 201
assert respuesta.json()["participante_id"] == "P-001"

```

Criterio para avanzar

- ☐ La API tiene contrato, versión y errores consistentes.
- ☐ Las reglas del dominio no dependen del servidor web.
- ☐ La base de datos se crea mediante migraciones.
- ☐ Las pruebas no dependen del orden de ejecución.
- ☐ Las fechas, unidades y procedencia están explícitas.

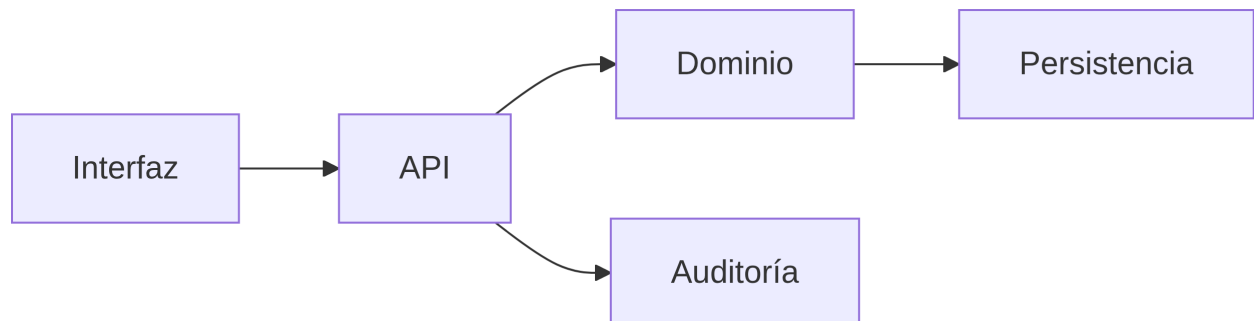
Nivel 4. Software preparado para operar

Meta: desplegar el sistema de forma repetible, detectar fallos y reducir riesgos de seguridad.

Duración sugerida: semanas 24 a 30.

Arquitectura antes que infraestructura

Para la mayoría de proyectos académicos y primeras versiones, un **monolito modular** es una mejor decisión que varios microservicios. Reduce despliegues, comunicación distribuida, monitoreo y fallos de red. Los microservicios se justifican cuando existen límites de dominio claros, equipos independientes o necesidades de escalado diferentes.



Contenedor reproducible

```

FROM ghcr.io/astral-sh/uv:0.12.0 AS uv_source

FROM python:3.13-slim AS runtime

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

```

```

WORKDIR /app

COPY --from=uv_source /uv /uvx /bin/
COPY pyproject.toml uv.lock ./
RUN uv sync --locked --no-dev --no-install-project

COPY src ./src
RUN uv sync --locked --no-dev

USER 10001

EXPOSE 8000
CMD ["/app/.venv/bin/uvicorn", "saludlocal.api.main:app", "--host", "0.0.0.0", "--port", "8000"]

```

En un sistema real también se debe:

- fijar imágenes mediante su resumen criptográfico;
- escanear dependencias e imagen;
- generar un inventario de componentes o SBOM;
- ejecutar con usuario sin privilegios;
- mantener secretos fuera de la imagen;
- limitar CPU, memoria y acceso a red; y
- documentar el proceso de actualización y reversión.

Seguridad desde el diseño

La versión 2025 del [OWASP Top 10](#) mantiene el control de acceso roto como el riesgo principal e incluye los fallos de cadena de suministro entre las categorías centrales. No basta con “agregar seguridad” antes del despliegue.

Use este conjunto mínimo:

1. autenticación y autorización separadas;
2. mínimo privilegio por rol y operación;
3. validación de entradas y codificación segura de salidas;
4. cifrado en tránsito y protección adecuada en almacenamiento;
5. secretos administrados fuera del repositorio;
6. dependencias bloqueadas, revisadas y actualizadas;
7. registros de auditoría protegidos contra alteración;
8. copias de seguridad con pruebas de restauración;
9. análisis de amenazas antes de desarrollar funciones críticas; y
10. procedimiento de respuesta y reversión.

Regla para los registros: un *log* debe permitir investigar un fallo sin convertirse en una copia secundaria de la historia clínica. Registre identificadores técnicos, eventos y resultados; evite nombres, documentos, diagnósticos, tokens y contenido clínico innecesario.

Observabilidad

La observabilidad permite inferir el estado interno de un sistema a partir de sus salidas. [OpenTelemetry](#) propone un marco neutral para producir y exportar trazas, métricas y registros.

Señal	Qué permite observar	Ejemplo
Métrica	Comportamiento agregado	Latencia p95 y tasa de errores

Señal	Qué permite observar	Ejemplo
Registro	Evento con contexto	Fallo de validación sin dato sensible
Traza	Recorrido de una solicitud	API → servicio → base de datos

Para SaludLocal defina al menos:

- número de solicitudes por estado HTTP;
- latencia mediana y percentil 95;
- errores de base de datos;
- intentos de acceso no autorizado;
- cola o atraso de procesamiento; y
- versión de la aplicación desplegada.

Criterio para avanzar

- ☐ El despliegue se reproduce desde código versionado.
- ☐ Existe un análisis básico de amenazas.
- ☐ Las dependencias y componentes son inventariables.
- ☐ Los registros no exponen información sensible.
- ☐ Una alerta conduce a un procedimiento de diagnóstico.
- ☐ Existe una forma probada de volver a la versión anterior.

Nivel 5. Ingeniería de software para salud digital

Meta: incorporar interoperabilidad, seguridad clínica, trazabilidad y gestión del ciclo de vida.

Duración sugerida: semanas 31 a 35.

Interoperabilidad no significa intercambiar cualquier JSON

Dos sistemas son interoperables cuando intercambian información y conservan un significado compartido. Esto exige estructura, terminología, identificadores, perfiles, versiones y reglas de validación.

- [HL7 FHIR](#) organiza el intercambio de información de salud mediante recursos y APIs.
- [DICOM](#) es el estándar internacional para imágenes médicas e información relacionada.
- [DICOMweb](#) ofrece servicios web para almacenar, buscar y recuperar estudios e imágenes.

Una correspondencia inicial para SaludLocal podría ser:

Concepto local	Posible recurso FHIR	Observación
Participante	Patient	Debe seguir el perfil institucional o nacional
Medición	Observation	Código, valor, unidad, sujeto y fecha son esenciales
Fuente de medición	Device	Incluye identificación y metadatos del dispositivo
Procedencia	Provenance	Explica quién, cómo y cuándo produjo el dato
Episodio	Encounter	Aporta contexto asistencial

Esta tabla orienta el modelado, pero no constituye una implementación conforme. Deben utilizarse las guías de implementación y validadores correspondientes.

Contexto colombiano

El Ministerio de Salud y Protección Social adoptó el Resumen Digital de Atención en Salud mediante la [Resolución 1888 de 2025](#), dentro de la Interoperabilidad de la Historia Clínica Electrónica. Sus [recursos técnicos de apoyo](#) incluyen una guía de implementación HL7 FHIR, manuales y colecciones para pruebas.

Esto tiene una consecuencia práctica: un desarrollador que trabaje en sistemas de salud en Colombia debe comprender contratos interoperables, autenticación, terminologías, validación y trazabilidad. No es suficiente conocer un *framework* web.

Los datos de salud requieren protección reforzada. La [Ley 1581 de 2012](#) establece el régimen general de protección de datos personales. En cada proyecto deben verificarse las obligaciones vigentes, las políticas institucionales y la finalidad autorizada para el tratamiento.

Gestión de riesgos y ciclo de vida

Las normas no sustituyen el juicio técnico, pero ofrecen una estructura común:

- [IEC 62304](#) define procesos del ciclo de vida del software de dispositivo médico.
- [ISO 14971](#) estructura la gestión de riesgos de dispositivos médicos durante su ciclo de vida.
- [ISO/TS 24971-2:2026](#) orienta la aplicación de gestión de riesgos a IA y aprendizaje automático.

Una trazabilidad mínima puede representarse así:

Peligro o problema	Control	Requisito	Prueba	Evidencia
Medición asociada al participante equivocado	Confirmar identidad técnica y contexto	RF-014	PI-014	Reporte de integración
Unidad incompatible	Validar código y sistema de unidad	RF-021	PU-021	Resultado automatizado
Acceso no autorizado	Autorizar por rol y propósito	RNF-008	PS-008	Registro de seguridad
Pérdida de trazabilidad	Auditoría inmutable y versionada	RNF-012	PA-012	Consulta de auditoría

Factores humanos

Una interfaz puede ser técnicamente correcta y aun así inducir errores. Evalúe:

- legibilidad y jerarquía de información;
- prevención de acciones irreversibles;
- confirmación de identidad y contexto;
- mensajes que expliquen cómo corregir el problema;
- comportamiento bajo carga de trabajo y presión de tiempo;
- accesibilidad; y
- diferencia visible entre dato original, dato calculado y recomendación.

Criterio para avanzar

- ☐ El propósito previsto del sistema está documentado.
- ☐ Cada riesgo importante tiene control y prueba.
- ☐ Los conceptos interoperables se validan contra una guía aplicable.
- ☐ La privacidad se considera desde la recopilación, no solo en el acceso.
- ☐ Profesionales del dominio participan en requisitos y validación.

Nivel 6. IA, sistemas avanzados y operación continua

Meta: integrar IA sin perder control sobre datos, cambios, desempeño y responsabilidad.

Duración sugerida: semanas 36 a 40 y aprendizaje continuo.

IA para desarrollar software

Los asistentes de IA pueden explicar un error, proponer pruebas, generar una primera versión o revisar un cambio. Sin embargo, producir más código no equivale a producir mejor software. El [informe DORA 2025](#) encontró que la IA actúa como amplificador: mejora equipos con prácticas sólidas y también amplifica problemas en sistemas de trabajo débiles.

Use este flujo:

1. escriba el problema, las restricciones y el criterio de aceptación;
2. solicite una propuesta pequeña y revisable;
3. exija pruebas normales, límite y adversas;
4. inspeccione cada cambio y cada dependencia agregada;
5. ejecute formato, análisis, tipos, pruebas y escaneo;
6. compare el resultado con el requisito, no con la respuesta de la IA; y
7. registre las decisiones que afecten arquitectura, seguridad o riesgo.

No entregue a un asistente externo datos de pacientes, credenciales, llaves, secretos, información institucional restringida ni código cuya política impida compartirlo.

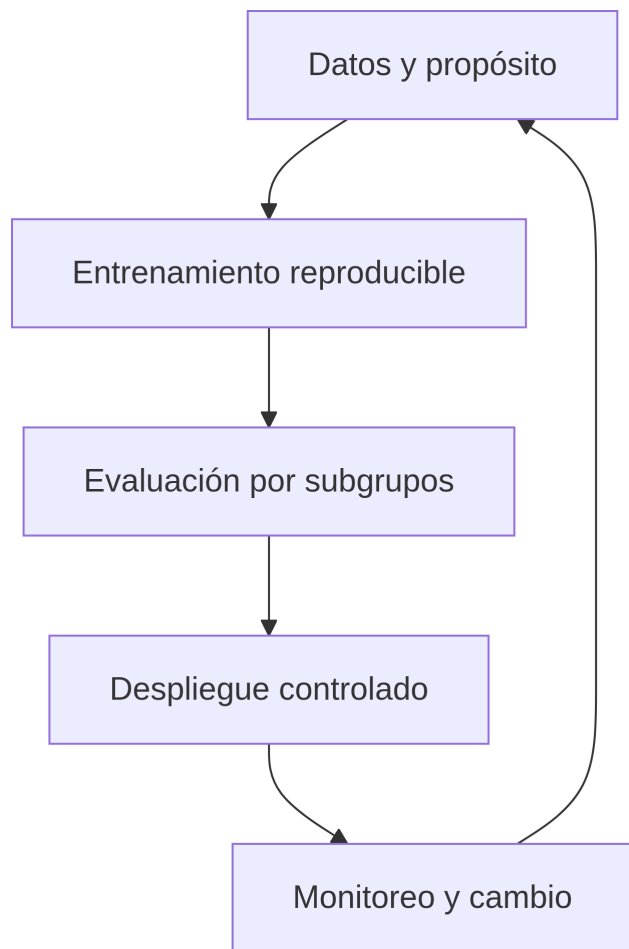
Ejemplo de solicitud técnica a un asistente

Implementa únicamente la validación del campo `unidad` en `MedicionCreate`. Acepta los códigos definidos en la constante `UNIDADES_PERMITIDAS`; rechaza cualquier otro valor con un error de dominio. No cambies la API pública. Agrega pruebas para valor válido, mayúsculas no permitidas, cadena vacía y valor desconocido. Explica los supuestos y no agregues dependencias.

La solicitud limita el alcance, define evidencia y reduce cambios innecesarios.

Cuando la IA forma parte del producto

Un modelo entrenado no es un archivo estático. Depende de datos, código, configuración, infraestructura y contexto de uso.



El expediente técnico debería conservar:

- propósito previsto y población objetivo;
- origen, calidad y restricciones de los datos;
- separación de participantes entre entrenamiento, validación y prueba;
- versión del código, datos, parámetros y entorno;
- métricas de discriminación, calibración y utilidad clínica;
- desempeño por subgrupos pertinentes;
- análisis de errores y casos fuera de distribución;
- límites conocidos y condiciones de abstención;
- supervisión humana y procedimiento de escalamiento;
- monitoreo de deriva y desempeño real; y
- plan de actualización, aprobación y reversión.

La [FDA](#) ha reforzado el enfoque de ciclo de vida total para funciones de dispositivos habilitadas por IA. La OMS también recomienda que la IA para salud mantenga la ética, los derechos y la responsabilidad en el centro de su diseño y uso, incluida su [guía para modelos multimodales](#).

Modelos generativos y RAG en salud

Un sistema de generación aumentada con recuperación o RAG puede consultar una base documental antes de responder. Esto mejora la trazabilidad de fuentes, pero no garantiza que la respuesta sea correcta.

Debe evaluarse:

- calidad y vigencia de los documentos recuperados;
- cobertura de preguntas reales;
- citas que correspondan con la afirmación;
- resistencia a instrucciones maliciosas dentro de documentos;
- manejo de preguntas sin evidencia suficiente;
- filtración de información sensible;
- consistencia entre versiones; y
- posibilidad de revisión y corrección humana.

En un flujo clínico, la opción “no tengo evidencia suficiente” puede ser más segura que una respuesta fluida pero no sustentada.

Criterio de finalización

- ☐ La IA tiene un propósito y un límite verificables.
- ☐ Los datos y experimentos son reproducibles.
- ☐ La evaluación incluye subgrupos, calibración y análisis de error.
- ☐ Existe supervisión humana con responsabilidad definida.
- ☐ El desempeño se monitorea después del despliegue.
- ☐ Cada actualización tiene evaluación, aprobación y reversión.

Tendencias actuales que vale la pena comprender

Una tendencia es útil cuando reduce un problema medible. Adoptarla solo porque es nueva aumenta deuda técnica y carga cognitiva.

Tendencia	Valor posible	Riesgo frecuente	Primer paso razonable
Desarrollo asistido por IA y agentes	Acelera exploración, pruebas y tareas repetitivas	Código no verificado y dependencia excesiva	Exigir criterios de aceptación, revisión y pruebas
Ingeniería de plataformas	Reduce fricción mediante rutas de despliegue comunes	Construir una plataforma mayor que las necesidades	Estandarizar una plantilla y un flujo de CI
DevSecOps y cadena de suministro	Detecta vulnerabilidades desde el cambio inicial	Acumular alertas sin priorización	Bloqueo de dependencias, SBOM y política de actualización
Observabilidad con OpenTelemetry	Facilita explicar fallos distribuidos	Recopilar datos sin preguntas operativas	Definir métricas y trazas para un flujo crítico
Arquitecturas orientadas a eventos	Desacopla procesos y soporta trabajo asíncrono	Duplicados, orden incierto y depuración difícil	Diseñar idempotencia y trazabilidad antes del <i>broker</i>
Interoperabilidad FHIR y DICOMweb	Conserva significado entre sistemas	Mapear campos sin semántica compartida	Partir de una guía de implementación aplicable
IA en el borde	Reduce latencia y exposición de datos	Dispositivos heterogéneos y modelos desactualizados	Medir recursos, precisión y actualización segura
Datos sintéticos y aprendizaje federado	Puede reducir transferencia de datos sensibles	Confundir menor exposición con anonimato garantizado	Evaluar utilidad, privacidad y riesgo de reidentificación
Ingeniería de evaluación para IA generativa	Hace medible calidad, seguridad y regresiones	Usar ejemplos pequeños o no representativos	Crear un conjunto versionado de casos reales y adversos

Tendencia	Valor posible	Riesgo frecuente	Primer paso razonable
Computación sostenible	Reduce costo y consumo innecesario	Optimizar sin medir el impacto dominante	Medir uso, latencia y frecuencia antes de cambiar

Adopción por etapas

Adoptar desde el inicio:

- control de versiones;
- entorno reproducible;
- tipos y validación;
- pruebas automatizadas;
- revisión de cambios;
- gestión de secretos;
- documentación del propósito y los límites; y
- datos sintéticos para aprendizaje.

Adoptar cuando exista una necesidad:

- contenedores;
- base de datos administrada;
- observabilidad distribuida;
- colas y eventos;
- infraestructura como código;
- plataforma interna; y
- despliegue en el borde.

No adoptar sin evidencia suficiente:

- microservicios para una aplicación pequeña;
- Kubernetes para un único servicio sin exigencias operativas;
- una base de datos diferente para cada tipo de dato;
- blockchain cuando una auditoría convencional resuelve el problema;
- un modelo generativo para una regla determinista; y
- formatos propios cuando existe un estándar aplicable.

Efectividad y eficiencia del desarrollador

Ser eficiente significa reducir trabajo innecesario. Ser efectivo significa entregar una solución que resuelva el problema correcto. La velocidad sin dirección produce retrabajo.

Hábitos de alto impacto

1. Trabajar en incrementos que puedan revisarse en menos de un día.
2. Limitar el trabajo simultáneo.
3. Automatizar una tarea solo después de comprenderla.
4. Corregir la causa del defecto, no únicamente su síntoma.
5. Escribir una prueba cuando un defecto sea reproducible.
6. Mantener las decisiones arquitectónicas en registros breves o ADR.
7. Preferir una solución simple hasta que la evidencia exija mayor complejidad.
8. Reservar tiempo para leer documentación oficial.
9. Explicar el código a otra persona: si no puede explicarse, todavía no se domina.
10. Medir resultados del sistema, no líneas de código ni horas frente al editor.

Indicadores del proceso

Indicador	Qué muestra	Señal de alerta
Tiempo desde cambio hasta despliegue	Flujo de entrega	Esperas largas por pasos manuales
Frecuencia de despliegue	Capacidad de entregar cambios pequeños	Entregas grandes y poco frecuentes
Tasa de cambios fallidos	Calidad del cambio	Reversiones y correcciones frecuentes
Tiempo de recuperación	Resiliencia	Diagnóstico lento o reversión incierta
Defectos escapados	Eficacia de controles	Los usuarios descubren problemas repetidos
Retrabajo	Claridad de requisitos y diseño	Se reescribe por falta de comprensión
Carga cognitiva	Facilidad de mantenimiento	Solo una persona entiende el sistema

Las métricas sirven para formular preguntas, no para castigar personas. Si se convierten en metas aisladas, tienden a ser manipuladas y pierden valor.

Plan de estudio de 40 semanas

Semanas	Enfoque	Producto verificable
1–2	Entorno, terminal, Git y uv	Repositorio reproducible
3–5	Tipos, funciones y condiciones	Validador de mediciones
6–8	Colecciones, archivos, errores y fechas	Interfaz de línea de comandos
9–11	Pruebas unitarias y casos límite	Suite inicial de pruebas
12–13	Formato, análisis estático y tipos	Puerta local de calidad
14–15	Git colaborativo e integración continua	Flujo automático en cada cambio
16–18	HTTP, REST, Pydantic y FastAPI	API documentada
19–21	SQL, repositorios y migraciones	Persistencia versionada
22–23	Pruebas de integración y contrato	Reporte de integración
24–26	Contenedores y configuración	Imagen reproducible
27–28	Seguridad y cadena de suministro	Modelo de amenazas y SBOM
29–30	Métricas, registros y trazas	Panel operativo mínimo
31–33	FHIR, DICOMweb y terminologías	Mapeo interoperable validado
34–35	Riesgo, usabilidad y trazabilidad	Matriz requisito-riesgo-prueba
36–38	IA reproducible y evaluación	Experimento versionado
39–40	Monitoreo, cambio y sustentación	Versión final y defensa técnica

Portafolio que demuestra capacidad

Un portafolio sólido no se limita a capturas de pantalla. Debe permitir comprender el problema, reproducir el sistema y examinar la evidencia.

Incluya:

- README.md con propósito, alcance, instalación y límites;
- arquitectura y contexto del sistema;
- requisitos y criterios de aceptación;
- registros de decisiones arquitectónicas;
- modelo de datos y diccionario;
- contrato de API;
- pruebas y reporte de resultados;
- análisis de amenazas;
- inventario de componentes;
- matriz de trazabilidad;
- procedimiento de despliegue y reversión;
- tablero de observabilidad;
- datos sintéticos y explicación de su generación;
- ficha o tarjeta del modelo si existe IA; y
- demostración breve del flujo completo.

Rúbrica de autoevaluación

Criterio	Peso	Pregunta principal
Problema y requisitos	15 %	¿La solución responde a una necesidad verificable?
Diseño y código	15 %	¿La estructura es clara y justificable?
Pruebas	15 %	¿Los casos importantes y los fallos están cubiertos?
Seguridad y privacidad	15 %	¿Los controles corresponden a riesgos concretos?
Interoperabilidad	10 %	¿El significado se conserva mediante estándares?
Despliegue y observabilidad	10 %	¿Puede operarse y diagnosticarse el sistema?
Reproducibilidad y documentación	10 %	¿Otra persona puede ejecutar y comprender el trabajo?
Límites y validación de salud	10 %	¿Se reconoce qué evidencia falta para uso real?

Lista final de verificación

Problema y usuarios

- ☐ El propósito previsto está escrito en una oración clara.
- ☐ Los usuarios y responsables están identificados.
- ☐ Cada función importante tiene criterio de aceptación.
- ☐ Los límites y usos no previstos están documentados.

Código y datos

- ☐ El entorno se reproduce desde archivos versionados.
- ☐ Las unidades, fechas, identificadores y valores ausentes son explícitos.
- ☐ Los datos de prueba son sintéticos o están correctamente autorizados.
- ☐ Las migraciones y cambios de esquema son trazables.

Calidad y seguridad

- ☐ Los controles automáticos se ejecutan antes de integrar.
- ☐ Existen pruebas para casos normales, límite, inválidos y adversos.
- ☐ Los secretos están fuera del repositorio.
- ☐ La autorización se verifica en el servidor.
- ☐ Las dependencias están bloqueadas e inventariadas.
- ☐ La restauración y la reversión se han probado.

Salud e IA

- ☐ Los riesgos importantes se relacionan con controles y pruebas.
- ☐ La interoperabilidad sigue una guía aplicable.
- ☐ La interfaz diferencia datos, cálculos y recomendaciones.
- ☐ El desempeño se evalúa por subgrupos pertinentes.
- ☐ Existe supervisión humana y un mecanismo de escalamiento.
- ☐ Los cambios del modelo siguen un proceso controlado.

Fuentes y documentación recomendada

Las herramientas cambian con rapidez. Antes de adoptar una versión o una práctica, revise la documentación oficial y registre la fecha de la decisión.

Desarrollo y operación

- [Python Tutorial](#)
- [uv: documentación](#)
- [pytest: primeros pasos](#)
- [FastAPI: tutorial](#)
- [GitHub Actions para Python](#)
- [Quarto Websites](#)
- [OpenTelemetry](#)
- [DORA: State of AI-assisted Software Development 2025](#)

Seguridad

- [OWASP Top 10:2025](#)
- [SLSA: seguridad de la cadena de suministro](#)

Salud digital

- [HL7 FHIR](#)
- [DICOM Standard](#)
- [DICOMweb](#)
- [Interoperabilidad de la Historia Clínica Electrónica en Colombia](#)
- [Recursos técnicos IHCE del Ministerio de Salud](#)
- [OMS: ética y gobernanza de IA para salud](#)
- [FDA: inteligencia artificial en software como dispositivo médico](#)

Siguiente paso

No intente completar todas las secciones al mismo tiempo. Inicie el nivel 0, defina el primer requisito de SaludLocal y produzca un incremento que otra persona pueda ejecutar. La capacidad avanzada se construye acumulando decisiones pequeñas, verificadas y bien explicadas.